

Step 1: Create Azure Storage Account

The screenshot shows the Microsoft Azure portal interface for a storage account named 'keshss2531'. The left sidebar contains a navigation menu with options like Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage, Containers, File shares, Queues, and Tables. The main content area is divided into 'Essentials' and 'Properties' sections. The 'Essentials' section displays key account details: Resource group (LabsKraft2027), Location (francecentral), Subscription (LabsKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), Disk state (Available), Performance (Standard), Replication (Locally-redundant storage (LRS)), Account kind (StorageV2 (general purpose v2)), Provisioning state (Succeeded), and Created date (26/8/2025, 10:41:50 am). The 'Properties' section shows 'Data Lake Storage' (Hierarchical namespace: Enabled) and 'Security' (Require secure transfer for REST API operations: Enabled).

Step 2: Create Container and Upload Raw Files

- Container name: `raw1`
- Uploaded files: Sales and Product data (CSV and JSON)

The screenshot shows the Microsoft Azure portal interface for the 'keshss2531' Storage account, specifically the 'Containers' page. The left sidebar is the same as in Step 1, with 'Containers' highlighted. The main content area shows a table of containers. At the top, there are buttons for '+ Add container', 'Upload', 'Refresh', 'Delete', 'Change access level', 'Restore containers', and 'Edit columns'. Below these is a search bar and a dropdown for 'Only show active containers'. The table lists two containers: '\$logs' and 'raw1'. Both are created on 26/8/2025, with 'raw1' created at 10:44:34 am. Both have a 'Private' anonymous access level and an 'Available' lease state.

	Name	Last modified	Anonymous access level	Lease state
☐	\$logs	26/8/2025, 10:42:17 am	Private	Available ...
☐	raw1	26/8/2025, 10:44:34 am	Private	Available ...



Search

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

+ Add Directory ↑ Upload ↻ Refresh 🗑 Delete 📄 Copy 📄 Paste 🔄 Rename 🔑 Acquire lease 🔑 Break lease 🛠 Edit columns

raw1

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Search blobs by prefix (case-sensitive)

Only show active objects

Showing all 2 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	products	26/8/2025, 10:49:06 am				...
☐	sales	26/8/2025, 10:49:18 am				...

Add or remove favorites by pressing Ctrl+Shift+F

Step 3: Create Azure Databricks Connector

- Establish connection between Databricks and Azure Storage.



keshconnector

Access Connector for Azure Databricks

Search

Refresh Delete

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Settings

Automation

Help

Essentials

Resource group ([move](#))

[LabsKraft2027](#)

Location

France Central

Subscription ([move](#))

[LabsKraft](#)

Subscription ID

d008c7cc-9795-4292-bf79-74ae52cda63d

Tags ([edit](#))

[Add tags](#)

State

Succeeded

Resource ID

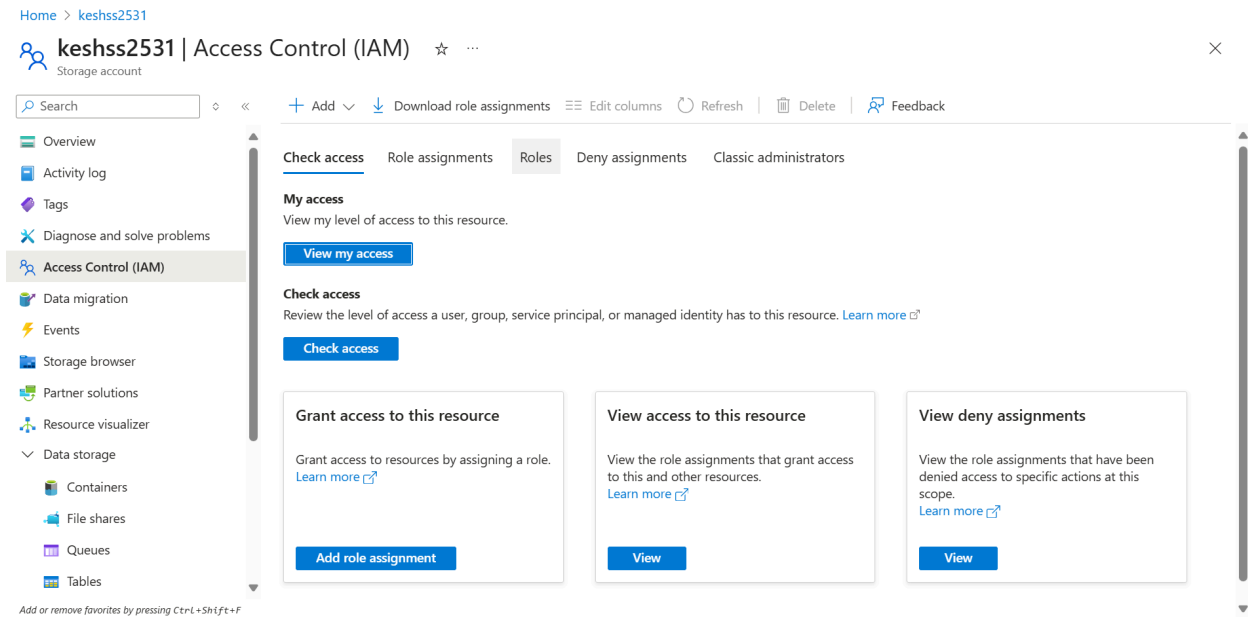
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

[JSON View](#)

Add or remove favorites by pressing Ctrl+Shift+F

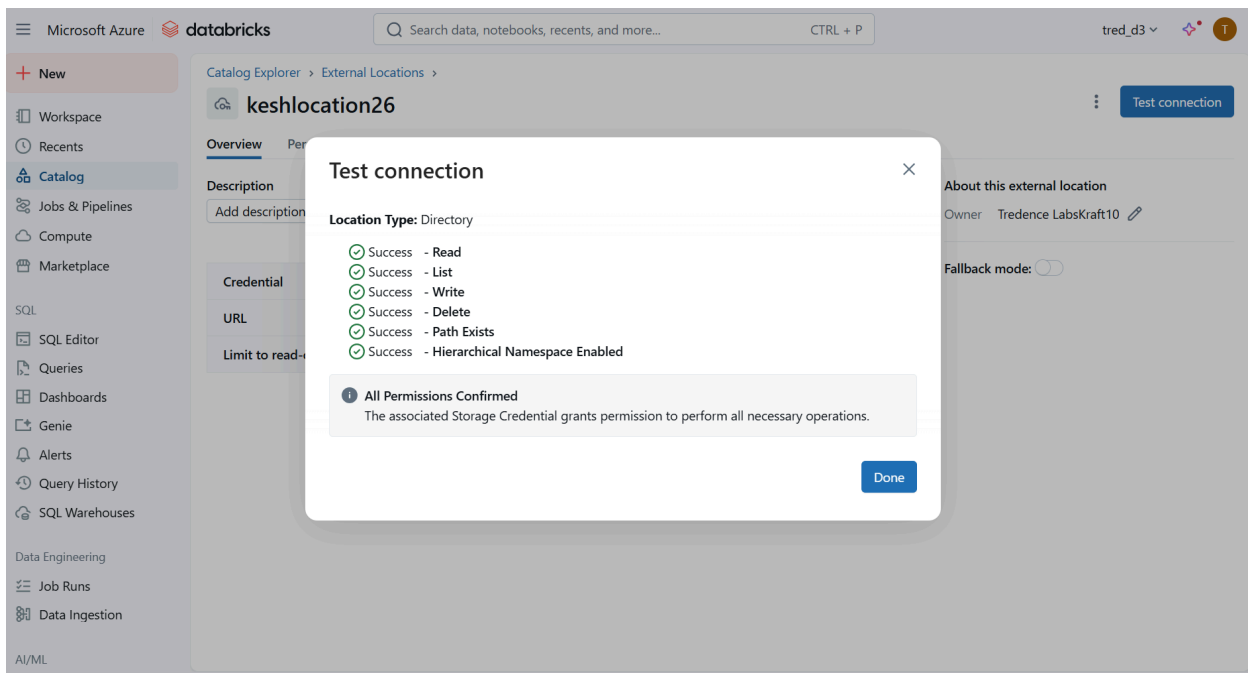
Step 4: Assign Roles

- Add necessary role assignments for access control.



Step 5: Create External Location in Catalog

- Define external location pointing to the raw container.



Step 6: Create Catalog

- Create a new catalog to organize tables.

The screenshot shows the Databricks interface with the 'Catalog Explorer' tab selected. The left sidebar shows the 'Catalog' section. The main panel displays the details for the 'keshcatalog26' catalog. The 'Overview' tab is active, showing the catalog's description, a filter for schemas, and a table of schemas.

Catalog Explorer > **keshcatalog26** [Share] [Create schema]

Overview Details Permissions Workspaces

Description

[AI generate] [Add]

Filter schemas 2 schemas

Name	Owner	Created at
default	tredence10@michaelcloudkraft...	Aug 26, 2025, 11:08 AM
information_schema	System user	Aug 26, 2025, 11:08 AM

About this catalog

Owner Tredence LabsKraft10 [edit]

Tags

[Add tags]

Step 7: Grant Permissions

- Grant access to all users or specific roles.

The screenshot shows the Databricks interface with the 'Catalog Explorer' tab selected. The left sidebar shows the 'Catalog' section. The main panel displays the details for the 'keshcatalog26' catalog. The 'Permissions' tab is active, showing a table of permissions granted to 'All account users'.

Catalog Explorer > **keshcatalog26** [Share] [Create schema]

Overview Details **Permissions** Workspaces

[Grant] [Revoke] Privileges Type to filter by principal

Principal	Privilege	Object
All account users	ALL PRIVILEGES	keshcatalog26
All account users	BROWSE	keshcatalog26
All account users	EXTERNAL USE SCHEMA	keshcatalog26
All account users	MANAGE	keshcatalog26

Step 8: Create Delta Live Tables (DLT) Pipelines

- ♦ **Bronze Layer (Raw Ingestion)**
- ♦ **Silver Layer (Cleaned Data)**
- ♦ **Gold Layer (Aggregated Insights)**

```
import dlt
from pyspark.sql.functions import col, to_date, sum, countDistinct
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType, DateType

# SALES SCHEMA
sales_schema = StructType([
    StructField("sale_id", StringType(), True),
    StructField("product_id", StringType(), True),
    StructField("region", StringType(), True),
    StructField("store_id", StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("sales_amount", DoubleType(), True),
    StructField("sale_date", StringType(), True)
])

# PRODUCTS SCHEMA (common structure for both CSV and JSON files)
products_schema = StructType([
    StructField("product_id", StringType(), True),
    StructField("product_name", StringType(), True),
    StructField("category", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("discount_rate", DoubleType(), True)
])

# -----
# BRONZE LAYER
# -----
```

```

@dlt.table(name="bronze_sales")
def bronze_sales():
    return (
        spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .schema(sales_schema)
            .load("abfss://raw1@keshss2531.dfs.core.windows.net/sales/")
    )

# Read Products CSV
@dlt.table(name="bronze_products_csv")
def bronze_products_csv():
    return (
        spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "csv")
            .schema(products_schema)
            .load("abfss://raw1@keshss2531.dfs.core.windows.net/products/*.csv")
    )

# Read Products JSON
@dlt.table(name="bronze_products_json")
def bronze_products_json():
    return (
        spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .schema(products_schema) # explicitly provide schema (fixes error)
            .load("abfss://raw1@keshss2531.dfs.core.windows.net/products/*.json")
    )

# -----
# SILVER LAYER (Cleaned)
# -----

@dlt.table(name="silver_sales")
@dlt.expect("Valid sales amount", "sales_amount >= 0")
@dlt.expect("Valid quantity", "quantity > 0")
def silver_sales():
    df = dlt.read("bronze_sales")

```

```

    return (
        df.dropna()
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sale_date", to_date("sale_date", "yyyy-MM-dd"))
    )

# Union both product sources
@dlt.table(name="silver_products")
def silver_products():
    csv_df = dlt.read("bronze_products_csv")
    json_df = dlt.read("bronze_products_json")
    return (
        csv_df.unionByName(json_df)
        .dropna()
        .dropDuplicates(["product_id"])
    )

# -----
# GOLD LAYER
# -----

@dlt.table(name="gold_sales_summary")
def gold_sales_summary():
    sales = dlt.read("silver_sales")
    products = dlt.read("silver_products")

    joined_df = sales.join(products, on="product_id", how="inner")

    result = (
        joined_df
        .withColumn("effective_price", col("price") * (1 -
col("discount_rate")))
        .withColumn("discount_amount", col("price") * col("discount_rate") *
col("quantity"))
        .groupBy("sale_date", "region", "product_id", "product_name",
"category")
        .agg(

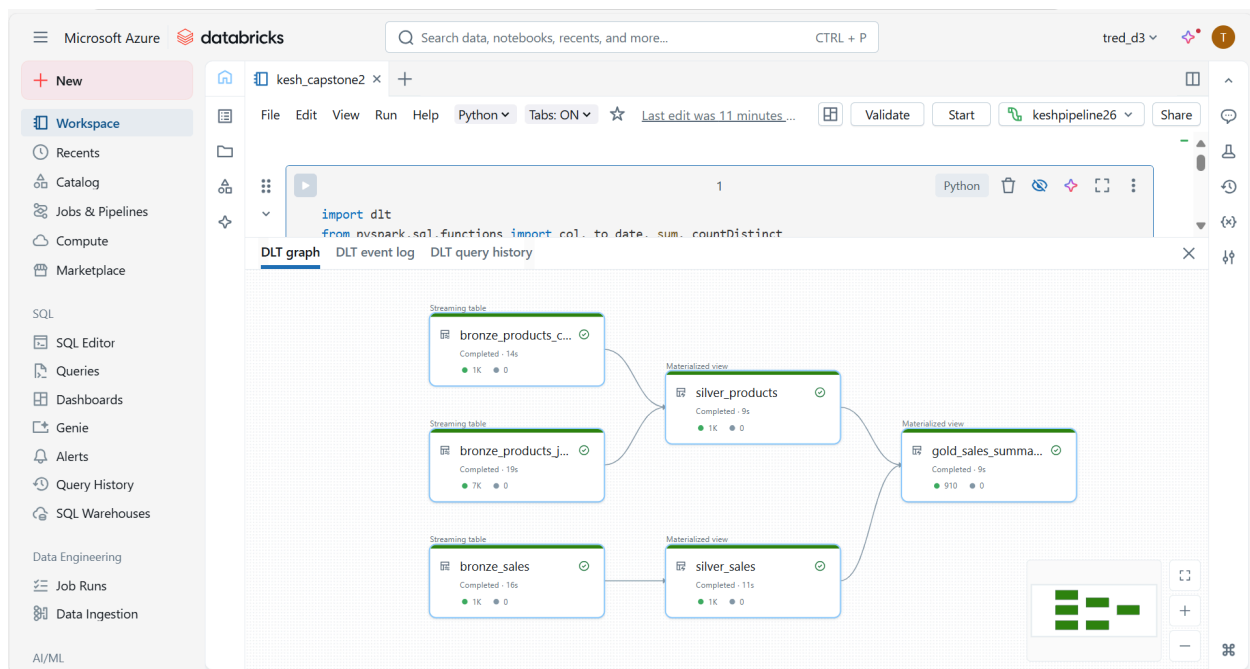
```

```

        sum("quantity").alias("total_units_sold"),
        sum("sales_amount").alias("total_revenue"),
        sum("discount_amount").alias("total_discount_given"),
        sum("effective_price").alias("total_effective_price")
    )
)

return result

```



Step 9: Introduce Schema Changes or Data Corruption

Modify the Sales CSV File

Upload the Corrupted File

Trigger the Pipeline

Observe Pipeline Behavior -> pipeline should log errors

Step 10: Connect Power BI to Azure Databricks and Visualize Data in Power BI

